Stress Test Pipeline — Test Results

Automated Test Documentation for Model Governance

MKM Research Labs

Version 1.0 — 04-April-2026

David K Kelly

Legal Notice

PROPRIETARY AND CONFIDENTIAL

This document, including all algorithms, models, methodology, formulas, calculations, and intellectual concepts contained herein, constitutes the exclusive intellectual property and confidential trade secrets of MKM Research Labs ("MKM").

Any usage, reproduction, distribution, modification, or disclosure of this document or any portion thereof, without the express prior written authorisation from MKM Research Labs is strictly prohibited and constitutes an infringement of intellectual property rights.

The document is provided on an "as is" basis. MKM Research Labs makes no representations or warranties of any kind, express or implied, regarding its accuracy, reliability, or suitability for any particular purpose.

1 Test Results — Stress Test Pipeline (MKM-ST-001)

Tests run on **04 April 2026 at 18:43:46**.

Table 1: Test Results Summary — Stress Test Pipeline (MKM-ST-001)

Total Tests	107
Passed	107
Failed	0
Skipped	0
Duration	23.82s

Table 2: Detailed Test Results — Stress Test Pipeline

Test Description	Acceptance Criteria	Result	Time
All already trained	`result["trained"] == 0`; `result["skipped"] == 3`	Pass	0.002s
After the first gauge trains, ETA should appear in output.	`"ETA" in captured`	Pass	0.249s
Missing gauge json	—	Pass	0.000s
Missing sequences	—	Pass	0.002s
No valid gauges	`result["trained"] == 0`	Pass	0.002s
One gauge already trained, one succeeds, one fails.	`result["trained"] == 1`; `result["failed"] == 1` (+1 more)	Pass	0.161s
Successful training	`result["trained"] == 3`; `result["failed"] == 0` (+2 more)	Pass	0.256s
Training failure is caught	`result["trained"] == 0`; `result["failed"] == 3` (+1 more)	Pass	0.259s
Counts default to zero	`s["alert_sequences"] == 0`; `s["warning_sequences"] == 0` (+1 more)	Pass	0.001s
Num gauges	`s["num_gauges"] == 4`	Pass	0.000s
Num sequences	`s["num_sequences"] == 7`	Pass	0.001s
Returns required keys	`key in s`	Pass	0.001s
When one gauge's classifier fails, pipeline continues.	`"classifier failed" in out`; `result["num_gauges"] == 2`	Pass	0.530s
Dict format	`len(result) == 2`	Pass	0.000s
Empty list	`_extract_gauges({"flood_gauges": []}) == []`	Pass	0.000s
List format	`result == [_G1, _G2]`	Pass	0.000s
Missing key returns empty	`_extract_gauges({}) == []`	Pass	0.000s

Test Description	Acceptance Criteria	Result	Time
When gaugehd/ has baseline data, the 'Loaded' message appears.	`"gauge baselines from gaugehd" in out or "heuristic base ...`	Pass	0.463s
Gauge summary dir written	`sg_dir.is_dir()`	Pass	0.000s
Gauge summary gauge ids	`len(d["gauge_ids"]) == 3`	Pass	0.001s
Gauge summary num gauges	`d["num_gauges"] == 3`	Pass	0.000s
Gauge summary schema version	`meta.exists(); d["schema_version"] == SCHEMA_VERSION_SPATIAL`	Pass	0.001s
Gauge summary sequence count	`len(gauge_files) == 3`	Pass	0.000s
Sequences file loadable	`len(seqs) == 40`	Pass	0.001s
Sequences json written	`(gauge_dir / SEQUENCES_FILENAME).exists()`	Pass	0.000s
Summary json written	`(gauge_dir / SUMMARY_FILENAME).exists()`	Pass	0.000s
Alert count non negative	`full_run["alert_sequences"] >= 0`	Pass	0.000s
Elapsed positive	`full_run["elapsed_seconds"] > 0`	Pass	0.000s
Num gauges	`full_run["num_gauges"] == 3`	Pass	0.000s
Num sequences	`full_run["num_sequences"] == 40`	Pass	0.000s
Returns dict	`isinstance(full_run, dict)`	Pass	0.000s
Severe lte warning	`full_run["severe_sequences"] <= full_run["warning_sequenc...`	Pass	0.000s
Type counts present	`"type_counts" in full_run; sum(full_run["type_counts"].values()) == 40`	Pass	0.000s
Warning lte alert	`full_run["warning_sequences"] <= full_run["alert_sequences"]`	Pass	0.000s
Batch generator populates category	`all(s.intensity_category in valid for s in seqs)`	Pass	0.001s
From dict missing category defaults empty	`reconstructed.intensity_category == ""`	Pass	0.000s
From dict round trip	`reconstructed.intensity_category == s.intensity_category`	Pass	0.001s
Serialized sequences have category	`all(s.intensity_category in valid for s in seqs)`	Pass	0.001s
To dict includes intensity category	`"intensity_category" in d; d["intensity_category"] == s.intensity_category`	Pass	0.001s
Legacy sequence_gauge_summary.json is deleted after split-mode write.	`not legacy.exists()`	Pass	0.526s
Corrupt file skipped	`len(baselines) == 1; "GAUGE-OK" in baselines`	Pass	0.002s

Test Description	Acceptance Criteria	Result	Time
Empty directory	`_load_gaugehd_baselines(d) == {}`	Pass	0.000s
Missing directory	`_load_gaugehd_baselines(tmp_path / "nonexistent") == {}`	Pass	0.000s
Missing mean level skipped	`len(baselines) == 0`	Pass	0.001s
Multiple gauges	`len(baselines) == 3`	Pass	0.005s
Valid gaugehd files	`"GAUGE-001" in baselines; baselines["GAUGE-001"]["mean_level"] == 1.25` (+1 more)	Pass	0.001s
Sequence with malformed sequence_start doesn't crash.	`result["num_sequences"] == 5`	Pass	0.559s
Explicitly test malformed date by patching sequences post-generation.	`result["num_sequences"] == 3`	Pass	0.553s
No crash when gauge json absent	`isinstance(result, dict)`	Pass	0.003s
Num gauges zero without gauge json	`result["num_gauges"] == 0`	Pass	0.002s
Sequences still written without gauge json	`(tmp_path / SEQUENCES_FILENAME).exists()`	Pass	0.003s
gauge.json present but every record missing gauge_id → n_all == 0.	`isinstance(result, dict); result["num_gauges"] == 0` (+1 more)	Pass	0.003s
Sequences still written when all gauges invalid	`(tmp_path / SEQUENCES_FILENAME).exists()`	Pass	0.002s
count=2001 with verbose=False prints ETA line at i=2000.	`result["num_sequences"] == 2001; "2,000" in out or "ETA" in out`	Pass	2.588s
Base level derived from alert	`r["base_level"] == pytest.approx(3.5 * 0.35)`	Pass	0.000s
Flood alert	`_parse_gauge(_G1)["flood_alert"] == pytest.approx(3.5)`	Pass	0.000s
Flood warning	`_parse_gauge(_G1)["flood_warning"] == pytest.approx(4.6)`	Pass	0.000s
Gauge id	`_parse_gauge(_G1)["gauge_id"] == "GAUGE-test001"`	Pass	0.000s
Lat lon	`r["lat"] == pytest.approx(51.46); r["lon"] == pytest.approx(-0.30)`	Pass	0.000s
Missing alert returns none	`_parse_gauge(bad) is None`	Pass	0.000s
Missing gauge id returns none	`_parse_gauge(bad) is None`	Pass	0.000s
Missing lat returns none	`_parse_gauge(bad) is None`	Pass	0.000s
Severe warning	`_parse_gauge(_G1)["severe_warning"] == pytest.approx(5.5)`	Pass	0.000s
Valid record returns dict	`result is not None`	Pass	0.000s
Warning defaults when absent	`r is not None; r["flood_warning"] == pytest.approx(4.0 * 1.33)` (+1 more)	Pass	0.000s

Test Description	Acceptance Criteria	Result	Time
Baseline without monthly means	`result["base_level"] == 1.5; result["monthly_means"] is None`	Pass	0.000s
With baseline	`result["base_level"] == 1.25; result["monthly_means"] == {"01": 1.4}`	Pass	0.000s
Without baseline uses heuristic	`result["base_level"] == pytest.approx(3.5 * 0.35); result["monthly_means"] is None`	Pass	0.000s
Empty monthly means	`_seasonal_base_level({}, mean_level=1.0, month=6) == 1.0`	Pass	0.000s
Missing month falls back	`_seasonal_base_level(monthly, mean_level=1.0, month=3) ==...`	Pass	0.000s
No monthly means	`_seasonal_base_level(None, mean_level=1.0, month=6) == 1.0`	Pass	0.000s
With monthly means	`_seasonal_base_level(monthly, mean_level=1.0, month=1) ==...; _seasonal_base_level(monthly, mean_level=1.0, month=7) ==...`	Pass	0.000s
All have intensity category	`all(r["intensity_category"] in valid for r in records)`	Pass	0.000s
All have sequence id	`all("sequence_id" in r for r in records)`	Pass	0.000s
All have sequence type	`all(r["sequence_type"] in valid for r in records)`	Pass	0.000s
All have total precip	`all("total_precip_mm" in r for r in records); all(r["total_precip_mm"] > 0 for r in records)`	Pass	0.000s
Gauge file count matches num gauges	`len(gauge_records) == 3`	Pass	0.000s
Peak hours in valid range	`0 <= s["peak_hour"] <= 167`	Pass	0.000s
Peaks positive	`all(s["peak_m"] > 0 for s in seqs)`	Pass	0.000s
Per gauge records have alert flags	`all("alert" in s for s in seqs)`	Pass	0.000s
Per gauge records have peak	`all("peak_m" in s for s in seqs)`	Pass	0.000s
Per gauge records have peak hour	`all("peak_hour" in s for s in seqs)`	Pass	0.000s
Per gauge sequence count	`len(seqs) == len(records)`	Pass	0.000s
Severe implies warning	`s["warning"]`	Pass	0.000s
Every sequence that breached warning must also have breached alert.	`s["alert"]`	Pass	0.000s
Error message lists available ids	`"GAUGE-test001" in str(exc_info.value)`	Pass	0.001s
Invalid gauge id raises	—	Pass	0.001s

Test Description	Acceptance Criteria	Result	Time
Named file gauge ids	`d["gauge_ids"] == ["GAUGE-test002"]`	Pass	0.001s
Named file peaks length one	`all(len(r["peaks_m"]) == 1 for r in d["sequences"])`	Pass	0.000s
Named file schema version	`d["schema_version"] == SCHEMA_VERSION_SPATIAL`	Pass	0.013s
Named file sequence count	`len(d["sequences"]) == 40`	Pass	0.000s
Named output file written	`(gauge_dir / "sequence_gauge_GAUGE-test002.json").exists()`	Pass	0.000s
Summary num gauges is one	`single_run["num_gauges"] == 1`	Pass	0.000s
Single gauge classifier result dict	`isinstance(result, dict)`	Pass	7.194s
Single gauge + train_classifier=True covers lines 353-361.	`result["num_gauges"] == 1; stressm_dir.exists()`	Pass	7.717s
Base level matches gauge params	`responses["GAUGE-A"][0]["base_level_m"] == pytest.approx(...; responses["GAUGE-B"][0]["base_level_m"] == pytest.approx(...`	Pass	0.000s
Gauge a values	`r["base_level_m"] == pytest.approx(1.5); r["peak_level_m"] == pytest.approx(5.0)` (+1 more)	Pass	0.000s
Gauge b values	`r["base_level_m"] == pytest.approx(2.0); r["peak_level_m"] == pytest.approx(7.0)` (+1 more)	Pass	0.000s
Level change is peak minus base	`r["level_change_m"] == pytest.approx(expected)`	Pass	0.000s
Response has base level m	`"base_level_m" in r`	Pass	0.000s
Response has level change m	`"level_change_m" in r`	Pass	0.000s
Creates new summary	`summary["num_gauges"] == 1; summary["num_trained"] == 1` (+1 more)	Pass	0.002s
Merges into existing	`summary["num_gauges"] == 2; summary["num_trained"] == 2` (+1 more)	Pass	0.003s
Replaces existing gauge	`summary["num_gauges"] == 1; summary["gauges"][0]["metrics"]["auc_roc"] == 0.95`	Pass	0.001s
Skipped gauge not counted as trained	`summary["num_trained"] == 0; summary["num_skipped"] == 1`	Pass	0.001s
With count ¡ 2000 and verbose=False, no mid-loop progress line.	`"ETA" not in out or "[" not in out`	Pass	1.074s

Test Description	Acceptance Criteria	Result	Time
count=501 with verbose=True triggers the progress line at i=500.	`result["num_sequences"] == 501; "[500" in out or "500/" in out`	Pass	1.628s

1.1 Test Document History

Date	Version	Description	Author
27-Mar-2026	1.0	Initial test suite (126 tests)	D. Kelly
04-Apr-2026	1.1	19 test(s) removed (total: 107)	D. Kelly